



Prompting for Agent Quality

Why optimizing quality beats cutting token costs

Eric W. Greene, Xebia

eric.greene@xebia.com



Table of contents

1 How LLMs, agents, and context windows work

You'll be able to explain why a long session costs more per turn and what compaction throws away

2 Model tier and relevant context

You'll have a rule for which tier to use and what to attach

3 Research, plan, implement

You'll see the loop beat a single prompt on the same task, repository and model

4 Guardrails and persistent instructions

You'll know what belongs in tests and linters and what belongs in an instruction file

5 Custom agents, skills, MCP servers, subagents

You'll be able to say which mechanism fits which problem

6 Power-user tips and analytics

You'll know where to read your spend, how to make fewer tool calls, and what to settle with your administrator

Your token bill is now real

GitHub Copilot moved to usage-based billing on June 1, 2026. Three things changed.

Tokens

Usage is now metered in input, output, and cached tokens, priced per model at the published rates. Code completions and next edit suggestions stay unmetered; chat, agent sessions, Copilot CLI, the cloud agent and code review draw on credits.

Output tokens are the most expensive of the three, and rework is mostly output.

AI Credits

Business and Enterprise seats include monthly credits equal to the seat price. Usage beyond that is billed at published rates, and it is on by default unless an administrator turns it off.

The promotional boosted credits ran June to August 2026; from September 2026, seats are at the standard allowance.

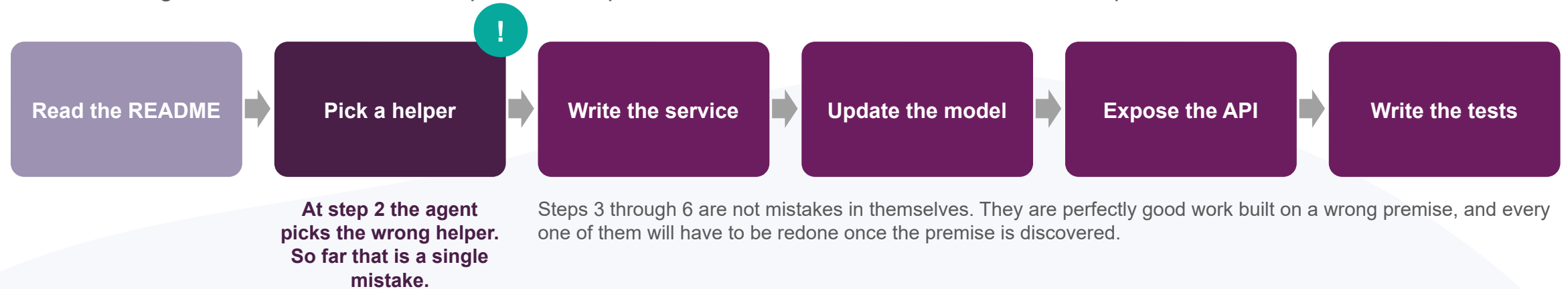
Budgets

Administrators can set budgets at the enterprise, cost center, organization, and user level, and cap spending once the credits are used up.

When a cap is reached, usage is blocked until the next billing cycle and nothing falls back to a cheaper model on its own. Whether to keep paid usage on is on the checklist near the end.

Small errors do not stay small

Here is one agent task broken into six steps. Each step is decided from a context that the earlier steps wrote.



What the rework loop actually costs

When the tests fail, the agent re-reads the code, decides again, and rewrites four files. Every rewrite is output tokens, which are the expensive kind, and every re-read sends the whole conversation back in as input. If we saved a little at step 2 by using a cheaper model or giving it less context, we made step 2 more likely to go wrong, and step 2 turns out to be the most expensive place to be wrong.

One runtime, three surfaces

VS Code

Agents in the editor, with session cost shown on screen

GitHub Copilot app

A desktop control center. Each session runs in its own git worktree, meaning a separate checkout of the same repository

Copilot CLI

The agent in your terminal, and where most of this course's vocabulary comes from

All three read the same instruction files, custom agents, and skills from the repository. You build them once and every surface picks them up.



Module 1

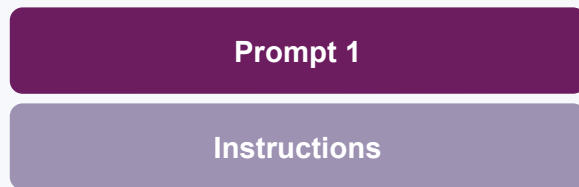
How LLMs, agents, and context windows work

A small error early in an agent task compounds through every step that follows. This module explains the mechanism behind it, and each idea here is shown in a demo.

The model is stateless

Every turn sends the whole conversation again. The model doesn't remember anything between calls; the client re-sends it all.

2 blocks sent this turn



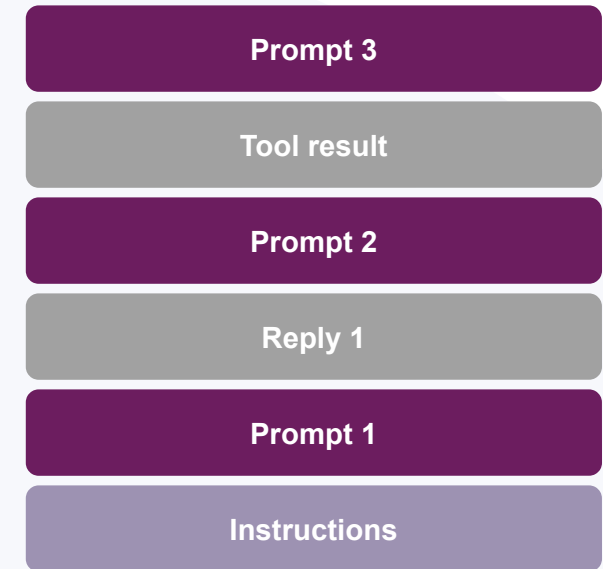
Turn 1

4 blocks sent this turn



Turn 2

6 blocks sent this turn



Turn 3

Three kinds of tokens

Input

What it is

Everything you send: the instructions, any attached files, the conversation so far, and every tool result.

Why it matters

This grows on every turn, and most of it is text you never typed yourself.

Output

What it is

Everything the model writes: explanations, code, plans, and the tool calls it decides to make.

Why it matters

The most expensive of the three per token. Rework is mostly output, plus the transcript re-sent as input.

Cached

What it is

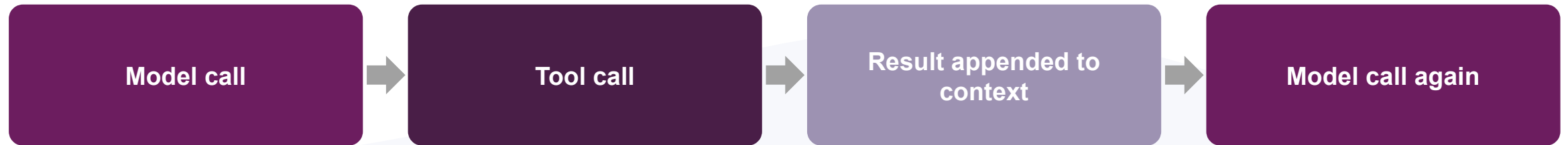
The repeated prefix the provider has already processed, such as stable instructions and the transcript so far.

Why it matters

The cheapest of the three. Because the transcript is re-sent every turn, most of a long session can be served from cache, as long as the prefix doesn't change.

An agent is a loop

Reading a file, running a command, or searching the code is a round trip, and the result of each one becomes input for the next.



What this means for cost

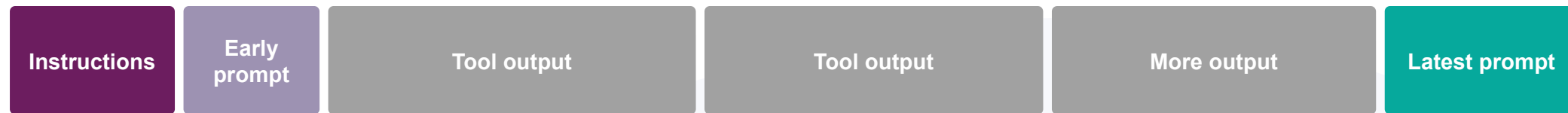
Every tool result is input you didn't write. Fifteen small file reads cost more than one well-targeted search, and the transcript then carries all fifteen of them for the rest of the session.

What this means for quality

The agent decides its next step from whatever the last result told it. A misleading README read at step 1 shapes what happens at step 2. Research, plan, implement exists to put a review point into this loop before any code is written.

Recency bias and context rot

As the window fills up, early instructions lose influence and the model's recall of specific details starts to degrade.



Stated in an early prompt: never change the public API

By turn 20, this is what carries the most weight

Recency bias

Later tokens carry more weight than earlier ones. A constraint you stated once at the start is competing with twenty turns of tool output by the time it matters. You can repeat what matters, or you can put it somewhere that's re-sent every turn, which is what the instruction file is for.

Context rot

Past a certain fill level, recall of specific facts degrades even though the tokens are still in the window. The million-token windows do not fix this, and they make the same failure more expensive, because the window is re-sent every turn, at cached rates for the unchanged prefix and full rates for everything new, and compaction or a changed prefix gives the discount away.

Compaction: what it keeps, what it loses

When the session gets close to capacity, it's summarized and the summary replaces the transcript. That happens automatically, and you can also trigger it yourself on either surface.

What survives

- The task and the decisions made so far
- Which files were touched, and why
- Open questions the summary noticed
- Instruction files, because they're re-sent every turn rather than remembered

What gets lost

- Exact tool output, such as line numbers and the precise error text
- The nuance in earlier reasoning
- Anything the summary judged unimportant at the time
- The cache benefit for the old prefix

Run `/compact yourself` in the CLI or in VS Code, and give it a focus instruction to say what the summary should keep. Left alone, it fires on its own as the session fills.

Demo: watching the window fill

VS Code

Copilot CLI

python-feequote, branch demo-start

What to look for

- The attached files are input tokens before the model has said anything
- Tool results land in the context and stay there
- What /compact kept and what it threw away
- Whether a rule from the first prompt survives /compact

Steps to reproduce: DEMO_SCRIPT.md in the demo kit



Module 2

Model tier and relevant context

The context window fills with whatever you attach, and compaction trims it when it gets full. The two levers you control before any of that happens are which model you pick and what you put in front of it.

Reading the price table

Every model has its own per-million-token rates. The columns matter more than the model names, which change often.

Column	What it counts	Relative cost	What pushes it up
Input	Everything sent on each turn	The baseline	Long transcripts, attached files, tool results, and re-sends after rework
Cached input	A prefix already processed	The lowest	Stable instructions and an unchanged transcript earn it; changes at the top of the prompt lose it
Cache write	Storing a new prefix	A little above input	Every time the prefix changes
Output	Everything the model writes	The highest, often several times input	Rework, long replies, and rewriting whole files instead of editing them

GitHub publishes a category for every model, lightweight through versatile to powerful, and the spread is wide: a lightweight model at \$0.20 in and \$1.20 out against a powerful one at \$10 and \$50. Some models are also tiered by context length, so the same token costs more once the window is large. That is one more reason not to let a session sprawl.

The decision rule

A lower tier is a good choice when the work is

- Mechanical: a rename, a signature change, or boilerplate
- Well specified: you could write the diff yourself if you had the time
- Verifiable: the tests or the compiler will catch anything it misses
- Local: one module, with callers you already know

A higher tier earns its cost when the work involves

- A design decision: which helper, which pattern, where it should live
- Unfamiliar or misleading code, such as stale docs and look-alike names
- Ambiguous requirements that deserve a question before any code
- Planning, because a plan runs to a few hundred words of output and the rework it saves you runs to thousands

For design tasks, count the whole cost, not only the credits: a cheaper model can retry several times for the price of one higher-tier run, but one defect that ships costs more than that. Automatic selection is a sensible default for anyone who would rather not think about this.

Relevant context, not all of it

1

Attach what you know matters

#file in VS Code and @file in the CLI put the files you already know are relevant in front of the model. A targeted attachment works better than letting the agent search, and much better than attaching the whole workspace.

2

Keep the prefix stable

Standing rules belong in the instruction file, where they are re-sent unchanged. A stable prefix earns cached tokens on every single turn.

3

Point at the tests

Tests are the most reliable description of behavior you have, though they encode bugs too. When the README and the tests disagree, start from the tests, and attach them.

4

Tune the picker

VS Code lets you set reasoning effort and context size per model. It's reasonable to spend more effort on the plan and less on the implementation.

Demo: the same task on two tiers

python-feequote, branch demo-start

What to look for

- Two sessions side by side, lower tier and higher tier, same task
- Both tiers find the retry helper; what differs is where each one puts the retry
- A hidden check showing which run charges the customer twice, then both /usage screens

Steps to reproduce: DEMO_SCRIPT.md in the demo kit



Module 3

Research, plan, implement

Modules 1 and 2 covered the window and the levers. This module puts a review point into the loop and measures what it does to the bill. Same repository, same prompt, same model, twice.

Why one-shot prompting compounds

One shot

Research, decide, and write, all in a single pass

- The first assumption is never reviewed by anyone
- Code exists before the approach has been checked
- Tests get written to match the code rather than the requirement
- The failure is discovered at the end, when everything has to change

Research, plan, implement

Research



Plan



Implement

- Research is read-only: find the tests and find the look-alike names
- The plan is something a person can review, so a wrong assumption is caught here, before any code
- Implementation is scoped to the plan and finishes with verification
- The plan is a short piece of output; the rework it prevents would have been a long one, plus the transcript re-sent as input

The difference between the two is the review point: a person, or at least a second look, between deciding and writing.

The loop, step by step

Research

Use read-only tools only. Find the tests that describe the behavior. Look for look-alike names and stale documentation. Ask the clarifying question now, while it's still cheap, rather than after the code exists. Copilot CLI has `/research` for exactly this phase, and it writes a report you can share.

Plan

Write a short plan: the files to touch, the approach, and the verification steps. Review it. If the approach is wrong, correcting one line of the plan is far cheaper than correcting four files of code.

Implement

Stay within the plan. Run the tests, the linter, and the type checker before calling the task done. If the plan turns out to need a change, change the plan first and then the code.

Writing a plan is cheap, and fixing one is cheaper still. Code is not cheap in either direction, and that is why the plan comes first.

Planning on each surface

VS Code: the Plan agent

- Pick Plan in the agent picker, or type /plan
- It uses read-only tools and asks clarifying questions
- It produces a plan with implementation and verification steps, saved to session memory
- Start Implementation hands off with the plan and the context intact

Copilot CLI: Plan mode

- Shift+Tab cycles through Standard, Plan, and Autopilot
- Plan mode explores and proposes before it touches any files
- Review the plan, then switch to Standard to implement
- Autopilot means no permission prompt per tool call, so it only makes sense once tests and instructions are in place

Copilot app: the plan canvas

- The plan appears as a canvas you can edit, reorder, approve, or redirect
- Each session runs in its own git worktree
 - Sessions can run locally or in a cloud sandbox hosted by GitHub
- The same plan discipline, applied to many sessions at once

Demo: run A, a single prompt in Copilot CLI

Copilot CLI

python-feequote, branch demo-start

What to look for

- Which files it reads before the first edit
- Whether legacy/fees gets imported; a total other than 3460 (usually 3459) means the wrong helper
- Whether it edits the failing test instead of the code
- Turns, tool calls, tokens, and credits from /usage

Steps to reproduce: DEMO_SCRIPT.md in the demo kit

Demo: run B, the Plan agent in VS Code

VS Code

python-feequote, branch demo-start

What to look for

- The clarifying question, if it asks one
- The plan names the correct helper before any code exists
- Acceptance totals of 3460 and 5000 on the first run
- Credits against run A, and how many files each run read before its first edit

Steps to reproduce: DEMO_SCRIPT.md in the demo kit



Module 4

Deterministic guardrails and persistent instructions

Run B needed a person to read the plan. This module is about getting run B's result without that person in the loop every time: checks the model can't argue with, and rules it sees on every turn.

Let the tooling do the checking

Tests

The most honest description of intended behavior you have. The agent can run them, read the failure, and fix its own work without waiting for you.

Linters

Style and hygiene rules the agent never has to be told about twice. Anything a linter enforces can come out of the instruction file.

Type checkers

These catch a wrong type or a changed signature before a single test runs. They won't catch a wrong helper with a valid signature; that's what the tests are for.

Builds

Treating warnings as errors turns a vague quality bar into a firm one that the agent can't talk its way past.

Autopilot modes only become safe once this loop is in place, and the loop still ends at a pull request that a person reviews.

An agent that runs the checks itself and reads the output is doing the review you would otherwise do by hand, and it's doing it at input-token prices.

Instruction files

File	Scope	When it loads	Notes
<code>.github/copilot-instructions.md</code>	The whole repository	Every request	Your standing rules. Keep it short, because it's re-sent every turn
<code>.github/instructions/*.instructions.md</code>	Paths matching <code>applyTo</code>	When matching files are in play	Frontmatter such as <code>applyTo: "tests/**"</code> (a glob, which is a path pattern)
<code>AGENTS.md</code>	The whole repository	Every request	Portable across tools; <code>CLAUDE.md</code> and <code>GEMINI.md</code> are read the same way
Organization instructions	Every repository in the organization	Every request	Set centrally. A default on GitHub.com, the cloud agent and code review; absent from VS Code chat, JetBrains chat and the CLI
Personal instructions	Your own sessions	Every request	Highest precedence where supported: Copilot CLI, JetBrains chat, GitHub.com chat. Not VS Code chat

Copilot CLI honors personal, repository, path-specific and agent files, though not organization instructions. VS Code chat honors the repository, path-specific and agent files. JetBrains chat honors personal, repository and path-specific instructions, and now runs the Copilot CLI harness. The support matrix in the takeaways is the current word, because it changes month to month.

Instructions that pay for themselves

Keep it short

Eight to twelve lines is plenty. Every line is re-sent on every turn, on every surface.

One rule per line, with the reason

For example: money is integer cents, because rounding errors here are visible to customers. The reason is what the model generalizes from.

Be concrete

Name the helper to use and the one to avoid. Point at the ticket that explains why.

Leave out what tools already enforce

If the linter catches it, the instruction is just wasted tokens.

Make the checks part of done

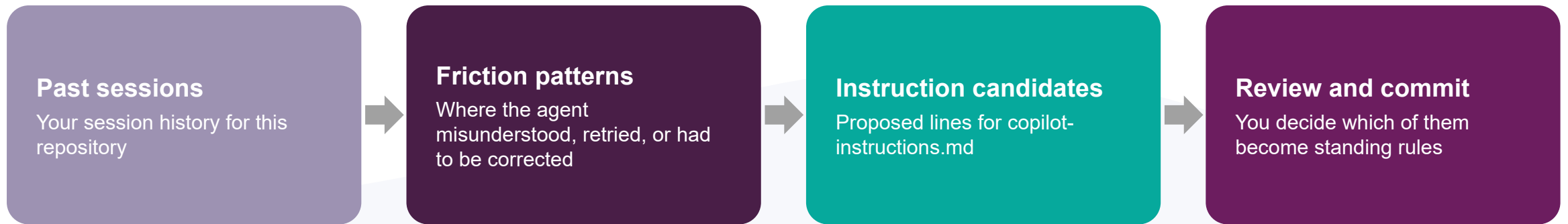
Run the tests, the linter, and the type checker before declaring a task complete. A task with failing checks isn't finished.

Change it rarely, through a pull request

A stable prefix is a cached prefix. CODEOWNERS on the .github folder keeps the file from drifting.

This is the one file where a little more writing makes every prompt that follows cheaper, which is why it's worth getting right.

/chronicle improve: instructions from evidence



/chronicle reads your session store: repository, branch, prompts, responses, the files that were touched, and any references. It runs in the CLI and in VS Code, though improve is CLI-only, and it now covers sessions from the CLI, VS Code, JetBrains, the app, code review and the cloud agent.

The improve subcommand is scoped to the current repository. Run it after a rough week and you get instruction lines grounded in what actually went wrong, rather than in what you imagine might go wrong.

Sessions sync to your GitHub account by default where your organization's policy allows it (on Business and Enterprise, an administrator enables it); remoteExport: false turns it off for you. More on what leaves your machine near the end.

Demo: green tests, the checks, and a failing test

Copilot CLI

VS Code

python-feequote, branch demo-checks, then main

What to look for

- Every test passes, yet the linter and type checker still find two problems
- Whether the agent runs the checks without being asked to in the prompt
- Updating expected values is right here because the requirement changed; in run A it wasn't
- The applyTo glob on the test conventions file, and what /chronicle improve proposes

Steps to reproduce: DEMO_SCRIPT.md in the demo kit



Module 5

Custom agents, skills, MCP servers, subagents

Instruction files are one of five ways to customize the agent. Most teams reach for the wrong one first, so this module starts with a table and ends with one file in the repository driving two surfaces.

Which mechanism, and when

Mechanism	What it is	Reach for it when	Where it lives
Custom instructions	Standing rules that are always in context	A rule applies to most of the work in the repository	copilot-instructions.md, .instructions.md, AGENTS.md
Agent skill	A packaged capability loaded on demand: SKILL.md plus scripts and resources	A multi-step procedure keeps coming back, such as release notes, a migration, or a house-style review	.github/skills/<name>/SKILL.md, ~/.copilot/skills/
Custom agent	A persona with its own instructions, tools, model, and allowed subagents	You want a specialist you can pick from the agent picker or delegate to	.github/agents/<name>.agent.md, ~/.copilot/agents/, organization level
MCP server (Model Context Protocol)	External tools and data the agent can call through a standard interface	The agent needs a system of record: issues, tickets, databases	CLI /mcp, VS Code mcp.json, organization allowlist
Subagent	An isolated context that does one sub-task and returns a result	A sub-task would flood the parent context, or could run in parallel	Automatic delegation; the VS Code agent tool; CLI delegation and /fleet

Three scenarios: which mechanism?

Which fits each one: an instruction, a skill, an agent, a subagent, or something else?

1

Every sprint, someone has to write release notes from the git log in the house format.

2

New code must never call the legacy fee helper, and it has to be caught every single time.

3

Before a refactor, someone needs to sweep forty files for every caller of a function and report back, without editing anything.

Three scenarios: the answers

The question to ask each time: does this have to happen every single time?

1

A skill

A recurring, multi-step procedure with a script and a house format behind it, like FeeQuote's release-notes skill.

2

A test, backed by an instruction

It has to be caught every time, so the architecture test enforces it. An instruction line keeps the agent from trying in the first place.

3

A subagent, or a read-only custom agent

The sweep stays out of the parent context and comes back as a list of findings, not forty files of raw content.

Skills load progressively

1 Metadata

The name and description from the frontmatter. Always in context, and only a few dozen tokens per skill.

2 Instructions

The body of SKILL.md. Loaded when the task matches or when you invoke the skill directly.

3 Resources

Scripts, examples, and reference files. Read only when the instructions point at them.

Why this matters

Twenty skills cost almost nothing until one of them is needed, which is the opposite of how a long instruction file behaves.

They're portable: VS Code, the CLI, and cloud sessions all read the same SKILL.md.

A script inside a skill turns a dozen tool calls into a single one. Review shared skills like any other dependency.

FeeQuote ships with a release-notes skill: one script and one house format, triggered simply by asking for release notes.

Anatomy of a custom agent

```
---
name: reviewer
description: Reviews a change against the FeeQuote
  conventions and the test suite. Read-only.
tools: ['read', 'search', 'codebase', 'usages',
  'problems', 'runTests', 'view', 'grep', 'glob']
model: claude-sonnet-5
agents: []
user-invocable: true
---
You are a code reviewer for the FeeQuote repository...
Report: correctness risks, convention violations, missing
tests, readability. Do not edit files. End with a verdict.
```

tools

Only what the persona needs. A reviewer gets read-only tools, so it can't edit anything even if someone asks it to. This is the pattern for any agent a bank approves. Two generations of name are in circulation: the cross-surface aliases GitHub publishes now (read, search, edit, execute) and the older per-surface names (codebase and usages in VS Code, view and grep in the CLI). Every surface ignores names it doesn't recognize, so one file can safely carry both.

model

A single model name, read the same way in VS Code, the CLI and the cloud agent; VS Code also accepts a prioritized list. Reviewers and planners belong on a higher tier than routine edits. Model IDs churn, so confirm yours resolves on your account.

agents

Which subagents this agent is allowed to invoke. An empty list means none, so there's no surprise delegation. This key is VS Code's; in the CLI, leave the agent tool out of the tools list.

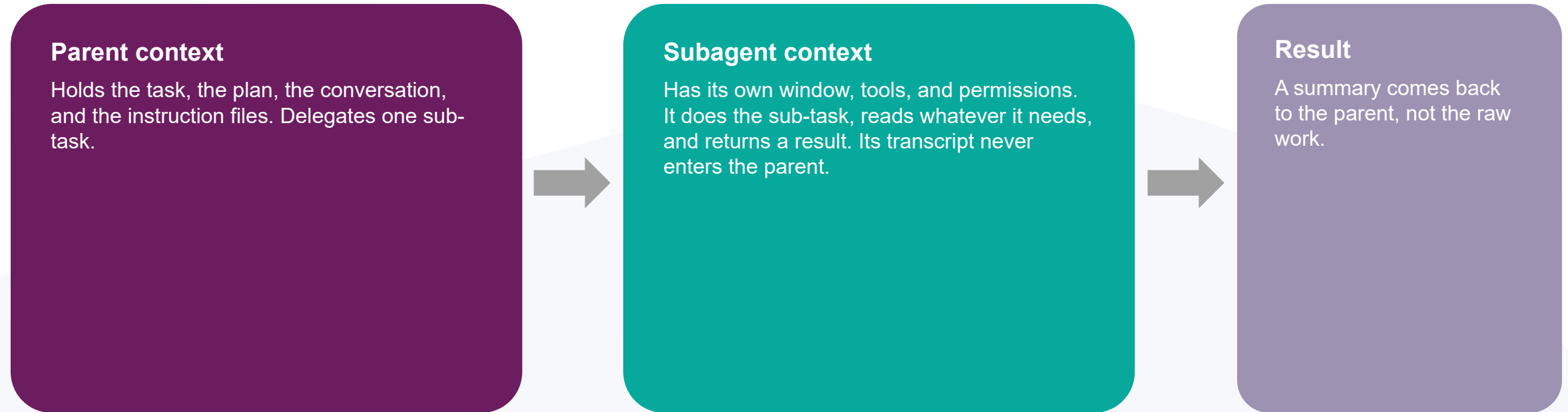
handoffs

Buttons that pass the conversation to another agent along with a prompt, for example from plan to implement. A VS Code key; the cloud agent ignores it.

Where it lives

.github/agents/ in the repository, ~/.copilot/agents/ for personal use, or organization-wide from the .github or .github-private repository. In Copilot CLI, user-level overrides repository, which overrides organization. Check the built-ins first: code-review, security-review, explore, research, task and more.

Subagents: an isolated context, at a real cost



- Delegation is automatic and probabilistic rather than guaranteed. Anything that has to happen every time belongs in a test, a linter, or an instruction.
- Each subagent has its own context, so the token count multiplies. The quality argument applies with more force here, not less.
- VS Code shows each subagent's model, elapsed time, active tool call, and credit usage, and that usage counts toward your session. The CLI delegates automatically and offers /fleet to run parts of one task in parallel, which buys wall-clock time with credits.

MCP servers: tools the agent didn't ship with

What

MCP is the Model Context Protocol, a standard way to give the agent external tools and data: issues and pull requests, tickets, databases, internal APIs, browsers. Copilot CLI ships with the GitHub MCP server built in.

Where

In the CLI: `/mcp show, add, auth, and search`. In VS Code: `mcp.json` in the workspace, with OAuth credentials kept in secure storage. Organizations can allowlist servers and publish a registry, and for a bank the allowlist should be enforced rather than advisory.

Cost

Every tool result comes back as input tokens. A server that returns a thousand rows when you needed one fills the window quickly, so prefer servers with narrow, well-described tools.

FeeQuote ships with a `.vscode/mcp.json` that configures the GitHub MCP server for VS Code. The CLI ignores that file and uses its own built-in `github-mcp-server` instead.

Demo: one file, two surfaces, a subagent

VS Code

Copilot CLI

Agents, MCP

python-feequote, branch main, with a dirty working tree

What to look for

- The same reviewer agent from the agent picker and from the terminal
- Whether the skill fires from a plain request; naming it is the fallback
- One tools line serving both surfaces, and why each surface ignores what it doesn't recognize
- The subagent's own tool calls, its result, and the credits it used

Steps to reproduce: DEMO_SCRIPT.md in the demo kit



Module 6

Power-user tips and analytics

Everything so far was about the quality of the first attempt. This last module is about reading your own spend. It comes in two halves: a tour of the power-user tips, and then spend and governance.

CLI tooling

Headless and scripted use

`copilot -p "..."` runs one prompt and exits, which suits scripts, CI, and hooks

`-s` gives clean output, just the agent's answer

`--allow-tool 'shell(git:*)'` and `--deny-tool` scope what a run is allowed to do. For pipelines, start from deny and allow only what the run needs, never with production credentials

`--agent reviewer` and `--model ...` pick the persona and the tier per run

`--share transcript.md` exports the session; inside a session `/share` defaults to a GitHub link, which is a data question for a bank

`/resume`, `/share`, `/usage`, `/context`, and `/compact` inside a session

Prefixes worth memorizing

! runs a shell command directly, with no agent round trip

`@file` attaches a file you already know matters

`#123` pulls in an issue or pull request by number

`&` delegates the prompt to a cloud session hosted by GitHub so you can keep working

`Shift+Tab` cycles Standard, Plan, and Autopilot

`Ctrl+Q` queues a message while the agent is busy

Collapsing tool calls, in two senses

On screen: reading less of them

Copilot CLI collapses the timeline by default.

Ctrl+O expands the recent items

Ctrl+E expands everything, on long sessions

Ctrl+T shows or hides the reasoning

VS Code groups tool calls and keeps them collapsed until you click. Either way it only changes the view: the calls already ran, so hiding them saves nothing.

On the bill: making fewer of them

Every tool call is a round trip that re-sends the context and appends a result.

- Attach the files you already know matter rather than letting the agent search for them
- Run commands you already know with the ! prefix
- Package repeated procedures as a skill with a script, so one call replaces a dozen
- Watch /context for the tool result share; /chronicle cost-tips reports on tool call frequency

/chronicle

Analytics over your sessions from the CLI, VS Code, JetBrains, the app, code review and the cloud agent. VS Code has every command here except improve, written with a colon: `/chronicle:cost-tips`.

Command	What it gives you	When to run it
<code>/chronicle standup</code>	What you worked on, grouped by branch and repository, with the related PRs and issues linked	Every morning, and paste it into the standup thread
<code>/chronicle tips</code>	Your usage patterns over the past week, with suggestions for what to change	Once a week
<code>/chronicle cost-tips</code>	Where the tokens went: prompt length, continuation steps, and tool call frequency	After a big task, or when the credits dashboard surprises you
<code>/chronicle search KEYWORD</code>	Finds the session where you touched that thing	Whenever you get the feeling you solved this before
<code>/chronicle improve</code>	Instruction candidates for this repository, drawn from past friction	After a rough week, then commit the ones you agree with
<code>/chronicle reindex</code>	Rebuilds the local store and syncs sessions to your account, where your organization's policy allows sync	When sessions are missing from search

What leaves your machine, and who can override what

Feature	What leaves your machine	Who controls it
Every model request	The prompt, attached files, tool results, and instruction files go to the model through GitHub. This is inherent to using Copilot at all.	Your organization's Copilot policies and model policy
/chronicle session store	Stored locally and synced to GitHub where allowed: prompts, responses, tools used, files touched. User opt-out: remoteExport: false.	On Business and Enterprise, the organization's Store local sessions in the Cloud policy; confirm retention and deletion
Cloud sessions (& prefix, cloud agent, app cloud sessions)	The repository, the prompt, and the work happen in a GitHub-hosted sandbox.	Organization settings for the cloud agent and the app
MCP servers	Tool calls and their results go to the server, which may be internal or third party.	The organization allowlist and registry; enforce it
Instruction and agent precedence	Personal overrides repository, which overrides organization, so instructions are a default. Organization instructions never reach VS Code chat, JetBrains chat or the CLI. Organization agents and skills, and enterprise managed settings, can be pinned.	Enterprise managed settings; then branch protection, required checks, CODEOWNERS on .github, and pull request review

Confirm each row with your administrator before rolling any of this out; slide 45 is the list of questions to ask.

Where to read your spend

Surface	Where to look	What it shows you
Copilot CLI	/usage and /context	AI credit usage, and the token breakdown by category for the active session
VS Code	The context window control in the chat input, the hover on a response footer, the Agents window, and the status dashboard	Session credits and cumulative tokens; input, cached input and output per turn by model; per-subagent usage; and the billing-cycle total on Business and Enterprise plans
GitHub Copilot app	The session view, or otherwise the GitHub.com usage dashboard	Session activity and organization-level usage
GitHub.com	The Copilot usage and budgets pages	Enterprise, cost center, organization and user-level spend against budgets: your administrator's view

For a team, three numbers to baseline this month and read again next month

First-attempt green rate on agent tasks. Credits per merged pull request. Rework turns per task. Individual dashboards tell you where the tokens went; these three tell you whether the practices are working.

Ask your administrator

1. Which models does our model policy enable, and is automatic selection available?
2. Are budgets hard-capped at the user level? What happens to a running session when a cap is hit?
3. Paid usage beyond the included credits is on by default: do we keep it, cap it, or turn it off, and at which level are budgets set?
4. Which Copilot CLI version is deployed, and is /chronicle present? Is session sync to the cloud enabled for us, and with what retention?
5. Is the GitHub Copilot app enabled for us? Is /fleet allowed, and is local sandboxing on by default?

6. Are organization-level instructions or custom agents already in place that our repositories inherit?
7. Is there an MCP server allowlist or registry, and is it enforced?
8. Are we Enterprise Managed Users, and does our data policy allow /share links, gists, and session sync?
9. Who owns .github in each repository? Is there a CODEOWNERS entry and a review requirement for instruction changes?
10. Which surfaces reach our JetBrains users, and are organization agents and skills published to them?

Reading your own costs with `/chronicle cost-tips`

`/chronicle cost-tips` in the CLI, `/chronicle:cost-tips` in VS Code

What it reports

- Where the tokens went: prompt length, continuation steps, and tool call frequency
- Rework usually shows up as extra continuation steps and repeated tool calls
- It covers sessions from every surface, so a CLI session and a VS Code session can be compared
- Run it after a big task, or when the credits dashboard surprises you

Three things to take back to your desk

1

Get the first attempt right

Research, then plan, then implement. A plan costs a few hundred words; the rework it saves you costs thousands, plus the transcript re-sent on every turn.

2

Let the tooling do the checking

Tests, linters, type checkers, and instruction files. The most reliable quality signal is the one the model didn't generate itself.

3

Measure in your own billing units

/usage, the context window control, and /chronicle cost-tips for you; first-attempt green rate, credits per merged pull request, and rework turns for the team.

An assumption carried through several files is the six-step chain from the start of the session, and rework is what it costs. If you remember one thing, make it this: spend your tokens on getting the first attempt right, because the rework is where the money actually goes.

What you'll receive, and what to try today

You'll receive

The FeeQuote repository

Python, TypeScript, C#, Java with Spring Boot, and ASP.NET with React, with every customization file, a demo script and a parallel-sessions exercise.

Templates

copilot-instructions.md, a path-specific .instructions.md, AGENTS.md, reviewer.agent.md, and a SKILL.md skeleton with a script.

The research, plan, implement checklist

One page, with fields to compare a run with and without a plan.

The decision table and the administrator checklist

Which mechanism, when; the questions to settle before rolling out; and a rollout order.

Links

One page of links: billing, models and pricing, the instruction support matrix, the CLI references, /chronicle, and the VS Code docs.

Try this today

1. Run /context on a live session and see what the tool results cost.
2. Run /init in your busiest repository, then put what it writes through the six rules.
3. Run /chronicle improve on your busiest repository and read what it proposes.
4. Run one real task twice, with a plan and without, and compare /usage.

Questions

Put your questions in the chat

Ask about anything from today: the demos, your own repositories, or rolling this out to your team. The moderator reads them out. Any we don't reach are answered in the follow-up email.



Thank you

Eric W. Greene, Xebia
eric.greene@xebia.com

xebia.com

